

Cloud infrastructure

[Menu](#)[On this page](#)

Séance 4

Pods & deployments

Objectifs

- Maîtriser les objets *pod*
- Gérer les déploiements (*replica set*, *stateful set*, *daemon set*)
- Scalabilité
- *Port forwarding* pour voir le pod.

Remarque

Il est nécessaire de maîtriser les notions : **déclaratif** *versus* **impératif**.

Les tâches suivantes se réaliseront toujours de manière déclarative.

Tâche 1

Déployez un Pod de manière déclarative *via* votre accès distant (cfr. tâche 3 de la séance 2) et accédez à son contenu en utilisant la commande `kubect1 port-forward` .

Exigences
L'application dans le conteneur écoute sur le port TCP/5000.
Exposer ce port interne sur le port TCP/8080 de la machine locale.
Le conteneur doit être visible dans son navigateur.

Question

Quelle commande `kubect1` faut-il utiliser pour accéder à l'application Web dans le pod depuis votre console ?

Tâche 2

Un [deployment](#) est un objet Kubernetes qui gère la création et les mises à jour des Pods. Il garantit l'exécution du nombre souhaité de *replicas* et gère automatiquement les mises à jour et les *rollbacks*.

Un *deployment* crée un [replicaset](#) qui crée à son tour les *pods*.

Créez un manifeste YAML déclaratif pour déployer un *deployment* avec l'image container utilisée précédemment et accédez à son contenu en utilisant la commande [kubectl port-forward](#) .

Exemple d'un déploiement simple

yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:latest
```

Exigences

Le *deployment* doit être nommé `<shortname>-deploy` , ou `<shortname>` est un placeholder pour votre identifiant utilisateur.

Le *pod* doit avoir le label `app: demo` et ce label doit être utilisé dans le *selector*.

le *deployment* doit avoir 1 replica.

Questions

Que peut-on observer comme différence entre un *pod* créé individuellement — comme à la tâche précédente — d'un *pod* créé *via* un *deployment* ? Quel est le lien avec le *replicaset* ?

Question

Quelle commande `kubectl` faut-il utiliser pour accéder à l'application Web dans le *pod* — *via* le *deployment* — depuis votre console ?

Tâche 3

Faites varier les paramètres du déploiement de la manière suivante et réalisez quelques observations sur le comportement des *pods* ainsi que le *replicaset*.

Les [probes](#) définies sur un *pod* permettent à *kubernetes* d'en connaître le status.

Exigences
Le deployment doit maintenant avoir 3 replicas
Ajoutez le label <i>environment</i> ayant la valeur <i>labo</i> à votre <i>pod template</i>
Ajoutez les probes <i>liveness</i> et <i>readiness</i> sur <code>/liveness</code> et <code>/readiness</code> .

Question

Que peut-on observer sur les *replicaset* et les *pods* après l'application du changement de paramètre du déploiement ?

Note sur les probes

Les endpoints `/liveness` et `/readiness` retournent simplement `OK` et `READY` respectivement.

Pour observer plus clairement le comportement des probes lors du démarrage des *pods*, vous pouvez modifier les paramètres `initialDelaySeconds` (le temps avant la première vérification) ou `periodSeconds` (le temps entre chaque vérification) dans vos probes pour ralentir leur exécution et pouvoir visualiser l'état.

Nous reviendrons sur ces concepts plus tard. Actuellement, retenons qu'il faut des probes ; il faut que kubernetes puisse tester l'état de ses pods.

Tâche 4

Un [*statefulset*](#) gère des Pods qui sont basés sur une même spécification de conteneur. Contrairement à un *Deployment*, un *StatefulSet* maintient une identité pour chacun de ces Pods. Ces *Pods* sont créés à partir de la même spec, mais ne sont pas interchangeables : chacun a un identifiant persistant qu'il garde à travers tous ses *rescheduling* (reprogrammations).

Créez un manifeste YAML déclaratif pour déployer un *statefulset* avec l'image container utilisée précédemment et accédez à son contenu en utilisant la commande [`kubect1 port-forward`](#) .

Exigences
Le <i>statefulset</i> doit être nommé <code><shortname>-sts</code> , ou <code><shortname></code> est un placeholder pour votre identifiant utilisateur.
Le <i>pod</i> doit avoir le label <code>app: demo</code> et ce label doit être utilisé dans le <i>selector</i> .
Le <i>statefulset</i> doit avoir 1 replica.
Les probes <i>liveness</i> et <i>readiness</i> sur <code>/liveness</code> et <code>/readiness</code> doivent être définies.

Questions

Que peut-on observer comme différence entre un *pod* créé *via* un *deployment* d'un *pod* créé *via* un `_statefulset_` ?

Tâche 5

Faites varier les paramètres du *statefulset* de la manière suivante et réalisez quelques observations sur le comportement des *pods*.

Exigences
Le <i>statefulset</i> doit maintenant avoir 2 replicas
Ajoutez le label <i>environment</i> ayant la valeur <i>labo</i> à votre <i>pod template</i>

Question

Que peut-on observer sur les *statefulset* et les *Pods* après l'application du changement de paramètre du déploiement ?

Question

Quelle commande kubectl faut-il utiliser pour accéder à l'application Web dans le pod — *via* le *statefulset* — depuis votre console ?

Tâche 6

Reprenez la première tâche de la première séance et comparez les variable d'environnement avec un *pod* scheduler par un *deployment* ainsi que par un *statefulset*.

Question

Que peut-on observer comme variables initialisées et valeurs différentes entre les instances du même container ? Quel lien cela a-t-il avec la manière dont le container est déployé ?

Mise à jour le: lundi 13 octobre 2025 à 13:43

Previous page
[Séance 3](#)

Next page
[Séance 5](#)